

Assignment 02 :

TYPES OF SYSTEM CALLS :

1) Process Control System Calls

These system calls are used to create, execute, terminate, and manage processes.

eg: `fork()`, `execve()`, `exit()`, `wait()`,

2) File Management System Calls :

These systems calls are used to create, open, read, write, close, delete, and manage files.

eg: `open()`, `read()`, `write()`, `unlink()`.

3) Device Management System calls.

These systems calls control input/output devices and allocates or release device resources.

eg: `read()`, `write()`, etc.

4) Information Maintenance System calls.

These system calls obtain or modify system information such as process IDs, time, and files, attributes.

eg: `getpid()`, `getppid()`, `start()`.

5) Communication System calls :

These systems calls allows processes to exchange data and communication with each other.

eg: `pipe()`, `socket()`, `msgget()`, `shmget()`.

6) Memory management System calls :
used to allocate, deallocate & manage memory.
eg: `brk()`, `mmap()`, `munmap()`; etc

7) Protection & security System calls :
used to control access permission & process privileges.
eg: `access()`, `chmod()`, `chown()`; etc.

Definations :

1) `Fork()`

→ Creates a new child process by duplicating the calling [parent] process.

2) `vfork()`

Creates a child process that shares the parent's memory until it executes a new program or exits

3) `clone()`

Creates a new process or thread with shared resources.

4) `execve()`

Replaces the current process with a new program

5) `exit()`

Terminates a process after performing cleanup tasks.

6) `_exit()`

Immediately terminates a process without standard cleanup.

7) `wait()`

Waits until a child process finishes execution

8) `waitpid()`

Waits for a specific child process to terminate

9) `waitid()`

Waits for process state changes and returns detailed information.

10) `kill()`

Sends a signal to a process to control or terminate it.

11) `getppid()`

Returns the parent process ID.

12) `nice()`

Changes the scheduling priority of a process

13) `setpriority()`

sets the priority of a process.

14) `getpriority()`

Retrieves the priority of a process

15) `open()`

Opens an existing file for reading or writing.

16) `creat()` Creates a new file or truncates an existing file.

17) `close()`

Closes an open file.

18) `read()`

Reads data from a file or device.

19) `write()`

Writes data to a file or device.

20) `pread()`

Reads data from a specified file position without changing the file pointer.

21) `pwrite()`

Writes data to a specified file position without changing the file pointer.

22) `lseek()`

Changes the current position of the file pointer.

23) `stat()`

Returns information about a file.

24) `fstat()`

Returns information about an open file.

25) `lstat()`

Returns information about a symbolic link

26) `access()`

Checks whether a file can be accessed.

27) `unlink()`

Deletes a file from the file system.

28) `rename()`

Changes the name or location of a file.

29) `rmdir()`

Removes an empty directory.

30) `dup()`

Creates a duplicate of a file descriptor

31) `dup2()`

Duplicates a file descriptor to a specified descriptor.

32) `fcntl()`

performs various operations on a file descriptor

33) `fsync()`

Writes all modified file data to the disk

34) `fdatasync()` writes file data to the disk without updating unnecessary metadata.

35) `truncate ()`

Changes the size of a file using its name

36) `ftruncate ()`

Changes the size of an open file using its file descriptor.